

DATABASE ARCHITECTURE & TABLES SPEC

Shop Sathi Database Schema Documentation

Complete Specification of Collections, Table Schemas, Mongoose Models, Indexes, and Entity Relationships

Database Engine

MongoDB (NoSQL) + Mongoose ODM

Collections Count

8 Core Collections (Users, Products, Categories, Carts, Wishlists, Orders, Reviews, Coupons)

Target Platform

Shop Sathi E-Commerce Backend Service

Document Release

v1.0.0 (Standalone Database Specification)



Backend Schema Document

Project: Production-Ready E-Commerce Platform

Version: 1.0

Status: Planning

Date: July 2026

Phase: 5 of 9

Table of Contents

1. [Overview](#)
 2. [Database Configuration](#)
 3. [Schema Conventions](#)
 4. [User Schema](#)
 5. [Product Schema](#)
 6. [Category Schema](#)
 7. [Cart Schema](#)
 8. [Wishlist Schema](#)
 9. [Order Schema](#)
 10. [Review Schema](#)
 11. [Coupon Schema](#)
 12. [Database Indexes](#)
 13. [Schema Relationships](#)
 14. [Mongoose Model Files](#)
 15. [Seed Data](#)
-

1. Overview

The platform uses **MongoDB** (NoSQL) with **Mongoose** as the ODM. All collections are designed for:

- Fast reads via proper indexing
- Minimal data duplication (denormalized only where performance demands)
- Scalability — no joins, references used via `ObjectId`

Collections Summary

Collection	Model Name	Primary Purpose
<code>users</code>	<code>User</code>	Customers + Admins
<code>products</code>	<code>Product</code>	Product catalog
<code>categories</code>	<code>Category</code>	Product categories
<code>carts</code>	<code>Cart</code>	Per-user shopping cart
<code>wishlists</code>	<code>Wishlist</code>	Per-user wishlist
<code>orders</code>	<code>Order</code>	Placed orders
<code>reviews</code>	<code>Review</code>	Product reviews
<code>coupons</code>	<code>Coupon</code>	Discount coupons

2. Database Configuration

2.1 Connection Setup

```
// src/config/db.js
import mongoose from 'mongoose'

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGODB_URI, {
      dbName: 'ecommerce',
    })
    console.log(`MongoDB Connected: ${conn.connection.host}`)
  } catch (error) {
    console.error(`DB Connection Error: ${error.message}`)
    process.exit(1)
  }
}

export default connectDB
```

2.2 Mongoose Global Settings

```
// In app.js – before connecting
mongoose.set('strictQuery', true) // reject unknown fields
mongoose.set('toJSON', {
  virtuals: true,
  versionKey: false,
  transform: (_, obj) => {
    delete obj.__v
    return obj
  }
})
```

3. Schema Conventions

Convention	Rule
Field names	<code>camelCase</code>
Required fields	Marked with <code>required: [true, 'Error message']</code>
String trim	All string fields use <code>trim: true</code>
Timestamps	All schemas use <code>{ timestamps: true }</code>
Soft delete	<code>isActive</code> boolean flag — never hard delete products/categories
Slug generation	Auto-generated from <code>name</code> using <code>slugify</code>
Version key	Disabled globally (<code>__v</code> removed)
Passwords	Never returned — <code>select: false</code>
Enums	Array of allowed values, <code>lowercase</code> or <code>uppercase</code> enforced

4. User Schema

4.1 Full Schema Definition

```

// src/models/User.model.js
import mongoose from 'mongoose'
import bcrypt from 'bcryptjs'
import crypto from 'crypto'

const addressSchema = new mongoose.Schema(
  {
    label: { type: String, trim: true }, // "Home", "Work", "Other"
    fullName: { type: String, required: true, trim: true },
    phone: { type: String, required: true },
    street: { type: String, required: true, trim: true },
    city: { type: String, required: true, trim: true },
    state: { type: String, required: true, trim: true },
    pincode: { type: String, required: true },
    country: { type: String, default: 'India' },
    isDefault: { type: Boolean, default: false },
  },
  { _id: true }
)

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
      minLength: [2, 'Name must be at least 2 characters'],
      maxLength: [50, 'Name cannot exceed 50 characters'],
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email'],
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minLength: [8, 'Password must be at least 8 characters'],
      select: false, // never returned in queries
    },
    phone: {
      type: String,
      trim: true,
    },
    avatar: {
      url: { type: String, default: '' },
      publicId: { type: String, default: '' },
    },
    role: {
      type: String,
      enum: ['customer', 'admin', 'superadmin'],
    }
  }
)

```

```

    default: 'customer',
  },
  addresses: [addressSchema],
  isBlocked: {
    type: Boolean,
    default: false,
  },
  resetPasswordToken: { type: String, select: false },
  resetPasswordExpire: { type: Date, select: false },
},
{ timestamps: true }
)

/* — Pre-save hook: hash password — */
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next()
  this.password = await bcrypt.hash(this.password, 12)
  next()
})

/* — Instance method: compare password — */
userSchema.methods.comparePassword = async function (candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password)
}

/* — Instance method: generate password reset token — */
userSchema.methods.getResetPasswordToken = function () {
  const rawToken = crypto.randomBytes(32).toString('hex')
  this.resetPasswordToken = crypto.createHash('sha256').update(rawToken).digest('hex')
  this.resetPasswordExpire = Date.now() + 10 * 60 * 1000 // 10 minutes
  return rawToken
}

/* — Virtual: full avatar URL fallback — */
userSchema.virtual('avatarUrl').get(function () {
  return this.avatar?.url || `https://api.dicebear.com/7.x/initials/svg?seed=${this.name}`
})

const User = mongoose.model('User', userSchema)
export default User

```

4.2 Field Reference

Field	Type	Required	Notes
name	String	✓	2–50 chars
email	String	✓	Unique, lowercase
password	String	✓	Hashed, never selected
phone	String	✗	Optional
avatar.url	String	✗	Cloudinary URL
avatar.publicId	String	✗	For deletion
role	Enum	✗	customer / admin / superadmin
addresses	Array	✗	Embedded sub-documents
isBlocked	Boolean	✗	Default: false
resetPasswordToken	String	✗	SHA-256 hashed, select: false
resetPasswordExpire	Date	✗	10-min TTL

5. Product Schema

5.1 Full Schema Definition

```

// src/models/Product.model.js
import mongoose from 'mongoose'
import slugify from 'slugify'

const variantSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },           // "Size", "Color"
    options: [{ type: String }],                     // ["S","M","L"] or ["Red","Blue"]
  },
  { _id: false }
)

const specificationSchema = new mongoose.Schema(
  {
    key: { type: String, required: true },           // "Material"
    value: { type: String, required: true },         // "100% Cotton"
  },
  { _id: false }
)

const imageSchema = new mongoose.Schema(
  {
    url: { type: String, required: true },
    publicId: { type: String, required: true },
    isMain: { type: Boolean, default: false },
  },
  { _id: true }
)

const productSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Product name is required'],
      trim: true,
      minLength: [3, 'Name must be at least 3 characters'],
      maxLength: [200, 'Name cannot exceed 200 characters'],
    },
    slug: {
      type: String,
      unique: true,
      index: true,
    },
    description: {
      type: String,
      required: [true, 'Product description is required'],
    },
    shortDescription: {
      type: String,
      maxLength: [300, 'Short description cannot exceed 300 characters'],
    },
    price: {
      type: Number,
      required: [true, 'Price is required'],
    },
  },
  { _id: true }
)

```

```

    min:      [0, 'Price cannot be negative'],
  },
  originalPrice: {
    type: Number,
    min: [0, 'Original price cannot be negative'],
  },
  discount: {
    type: Number,
    min: 0,
    max: 100,
    default: 0,
  },
  category: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Category',
    required: [true, 'Category is required'],
    index: true,
  },
  brand: {
    type: String,
    trim: true,
  },
  sku: {
    type: String,
    unique: true,
    sparse: true, // allows multiple nulls
    trim: true,
    uppercase: true,
  },
  images: [imageSchema],
  variants: [variantSchema],
  specifications: [specificationSchema],
  tags: [{ type: String, lowercase: true, trim: true }],
  stock: {
    type: Number,
    required: [true, 'Stock quantity is required'],
    min: [0, 'Stock cannot be negative'],
    default: 0,
  },
  isActive: {
    type: Boolean,
    default: true,
    index: true,
  },
  isFeatured: {
    type: Boolean,
    default: false,
  },
  ratings: {
    average: { type: Number, default: 0, min: 0, max: 5 },
    count: { type: Number, default: 0, min: 0 },
  },
  createdBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
  },

```

```

    },
  },
  { timestamps: true }
)

/* — Pre-save: auto-generate slug — */
productSchema.pre('save', function (next) {
  if (this.isModified('name')) {
    this.slug = slugify(this.name, { lower: true, strict: true })
  }
  next()
})

/* — Pre-save: auto-calculate discount — */
productSchema.pre('save', function (next) {
  if (this.originalPrice && this.price && this.originalPrice > this.price) {
    this.discount = Math.round(((this.originalPrice - this.price) / this.originalPrice) * 100)
  }
  next()
})

/* — Virtual: stock status — */
productSchema.virtual('stockStatus').get(function () {
  if (this.stock === 0) return 'out_of_stock'
  if (this.stock < 10) return 'low_stock'
  return 'in_stock'
})

/* — Text search index — */
productSchema.index({ name: 'text', description: 'text', tags: 'text', brand: 'text' })

const Product = mongoose.model('Product', productSchema)
export default Product

```

5.2 Field Reference

Field	Type	Required	Notes
name	String	✓	3–200 chars
slug	String	auto	From name, unique
description	String	✓	Full HTML allowed
shortDescription	String	✗	Max 300 chars
price	Number	✓	Current selling price
originalPrice	Number	✗	MRP for strike-through
discount	Number	auto	Auto-calculated %
category	ObjectId	✓	Ref: Category
brand	String	✗	—
sku	String	✗	Auto uppercase, unique
images	Array	✗	Max 6, Cloudinary
variants	Array	✗	Size, Color, etc.
specifications	Array	✗	Key-value pairs
tags	Array	✗	Lowercase strings
stock	Number	✓	Min: 0
isActive	Boolean	✗	Soft delete flag
isFeatured	Boolean	✗	Show on home page
ratings.average	Number	auto	Updated on review save
ratings.count	Number	auto	Updated on review save

6. Category Schema

6.1 Full Schema Definition

```

// src/models/Category.model.js
import mongoose from 'mongoose'
import slugify from 'slugify'

const categorySchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Category name is required'],
      unique: true,
      trim: true,
      minLength: [2, 'Name must be at least 2 characters'],
      maxLength: [50, 'Name cannot exceed 50 characters'],
    },
    slug: {
      type: String,
      unique: true,
    },
    description: {
      type: String,
      trim: true,
    },
    image: {
      url: { type: String, default: '' },
      publicId: { type: String, default: '' },
    },
    parent: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Category',
      default: null, // null = top-level category
    },
    isActive: {
      type: Boolean,
      default: true,
    },
  },
  { timestamps: true }
)

/* — Auto-generate slug — */
categorySchema.pre('save', function (next) {
  if (this.isModified('name')) {
    this.slug = slugify(this.name, { lower: true, strict: true })
  }
  next()
})

/* — Virtual: isSubcategory — */
categorySchema.virtual('isSubcategory').get(function () {
  return this.parent !== null
})

```

```
const Category = mongoose.model('Category', categorySchema)
export default Category
```

6.2 Field Reference

Field	Type	Required	Notes
name	String	✓	Unique, 2–50 chars
slug	String	auto	From name
description	String	✗	—
image.url	String	✗	Cloudinary URL
image.publicId	String	✗	For deletion
parent	ObjectId	✗	Ref: Category (null = top-level)
isActive	Boolean	✗	Default: true

7. Cart Schema

7.1 Full Schema Definition

```

// src/models/Cart.model.js
import mongoose from 'mongoose'

const cartItemSchema = new mongoose.Schema(
  {
    product: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Product',
      required: true,
    },
    name: { type: String, required: true }, // Snapshot at add time
    image: { type: String, default: '' }, // Snapshot at add time
    price: { type: Number, required: true }, // Snapshot at add time
    quantity: {
      type: Number,
      required: true,
      min: [1, 'Quantity must be at least 1'],
      default: 1,
    },
    selectedVariants: {
      type: Map,
      of: String, // { size: "M", color: "Red" }
    },
  },
  { _id: true }
)

const cartSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
      unique: true, // One cart per user
    },
    items: [cartItemSchema],
    coupon: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Coupon',
      default: null,
    },
    discountAmount: {
      type: Number,
      default: 0,
    },
  },
  { timestamps: true }
)

/* — Virtual: item count — */
cartSchema.virtual('itemCount').get(function () {
  return this.items.reduce((total, item) => total + item.quantity, 0)
})

```

```
/* — Virtual: subtotal — */
cartSchema.virtual('subtotal').get(function () {
  return this.items.reduce((total, item) => total + item.price * item.quantity, 0)
})

const Cart = mongoose.model('Cart', cartSchema)
export default Cart
```

7.2 Design Decisions

Why snapshot name/image/price?

Product data can change after item is added to cart. We store a snapshot so the cart always shows the price the user saw when they added it. At checkout, we re-verify current price and stock.

8. Wishlist Schema

8.1 Full Schema Definition

```
// src/models/Wishlist.model.js
import mongoose from 'mongoose'

const wishlistSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
      unique: true, // One wishlist per user
    },
    products: [
      {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Product',
      }
    ],
  },
  { timestamps: true }
)

/* — Virtual: product count — */
wishlistSchema.virtual('count').get(function () {
  return this.products.length
})

const Wishlist = mongoose.model('Wishlist', wishlistSchema)
export default Wishlist
```

9. Order Schema

9.1 Full Schema Definition

```

// src/models/Order.model.js
import mongoose from 'mongoose'

const orderItemSchema = new mongoose.Schema(
  {
    product: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Product',
    },
    name: { type: String, required: true }, // Snapshot
    image: { type: String, default: '' }, // Snapshot
    price: { type: Number, required: true }, // Price at purchase
    quantity: { type: Number, required: true, min: 1 },
    selectedVariants: {
      type: Map,
      of: String,
    },
  },
  { _id: true }
)

const shippingAddressSchema = new mongoose.Schema(
  {
    fullName: { type: String, required: true },
    phone: { type: String, required: true },
    street: { type: String, required: true },
    city: { type: String, required: true },
    state: { type: String, required: true },
    pincode: { type: String, required: true },
    country: { type: String, default: 'India' },
  },
  { _id: false }
)

const statusHistorySchema = new mongoose.Schema(
  {
    status: { type: String, required: true },
    note: { type: String, default: '' },
    updatedAt: { type: Date, default: Date.now },
    updatedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  },
  { _id: false }
)

const orderSchema = new mongoose.Schema(
  {
    orderNumber: {
      type: String,
      unique: true,
    },
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
    },
  },
  { _id: true }
)

```

```

    index:    true,
  },
  items: [orderItemSchema],

  shippingAddress: shippingAddressSchema,

  paymentMethod: {
    type:      String,
    enum:      ['COD', 'ONLINE'],
    required: true,
  },
  paymentStatus: {
    type:      String,
    enum:      ['pending', 'paid', 'failed', 'refunded'],
    default: 'pending',
  },
  orderStatus: {
    type:      String,
    enum:      ['pending', 'processing', 'shipped', 'delivered', 'cancelled'],
    default: 'pending',
    index:    true,
  },
  statusHistory: [statusHistorySchema],

  coupon: {
    type:      mongoose.Schema.Types.ObjectId,
    ref:       'Coupon',
    default: null,
  },

  /* — Financials — */
  subtotal:      { type: Number, required: true },
  shippingCharge: { type: Number, default: 0 },
  taxAmount:      { type: Number, default: 0 },
  discountAmount: { type: Number, default: 0 },
  totalAmount:    { type: Number, required: true },

  /* — Dates — */
  estimatedDelivery: { type: Date },
  deliveredAt:        { type: Date },
  cancelledAt:        { type: Date },
  cancelReason:       { type: String, trim: true },
},
{ timestamps: true }
)

/* — Pre-save: generate order number — */
orderSchema.pre('save', async function (next) {
  if (!this.orderNumber) {
    const date = new Date()
    const ymd = `${date.getFullYear()}${String(date.getMonth()+1).padStart(2, '0')}${String(date.ge
    const count = await mongoose.model('Order').countDocuments() + 1
    const serial = String(count).padStart(4, '0')
    this.orderNumber = `ORD-${ymd}-${serial}`
  }
}

```

```

    next()
  })

  /* — Pre-save: push to status history on status change — */
  orderSchema.pre('save', function (next) {
    if (this.isModified('orderStatus')) {
      this.statusHistory.push({ status: this.orderStatus })
    }
    next()
  })

  /* — Virtual: item count — */
  orderSchema.virtual('itemCount').get(function () {
    return this.items.reduce((sum, item) => sum + item.quantity, 0)
  })

  const Order = mongoose.model('Order', orderSchema)
  export default Order

```

9.2 Order Number Format

ORD-YYYYMMDD-XXXX

Examples:

ORD-20260701-0001 (first order)
 ORD-20260701-0127 (127th order)

9.3 Financial Calculation Logic

```

// In order.controller.js – before creating order

const TAX_RATE          = 0.18           // 18% GST
const FREE_SHIPPING_THRESHOLD = 499      // Free shipping above ₹499

const subtotal          = items.reduce((sum, i) => sum + i.price * i.quantity, 0)
const discountAmount    = couponApplied ? calculateDiscount(coupon, subtotal) : 0
const taxableAmount     = subtotal - discountAmount
const taxAmount         = parseFloat((taxableAmount * TAX_RATE).toFixed(2))
const shippingCharge    = taxableAmount >= FREE_SHIPPING_THRESHOLD ? 0 : 99
const totalAmount       = taxableAmount + taxAmount + shippingCharge

```

10. Review Schema

10.1 Full Schema Definition

```
// src/models/Review.model.js
import mongoose from 'mongoose'

const reviewSchema = new mongoose.Schema(
  {
    product: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Product',
      required: true,
      index: true,
    },
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
    },
    rating: {
      type: Number,
      required: [true, 'Rating is required'],
      min: [1, 'Rating must be at least 1'],
      max: [5, 'Rating cannot exceed 5'],
    },
    title: {
      type: String,
      trim: true,
      maxLength: [100, 'Title cannot exceed 100 characters'],
    },
    comment: {
      type: String,
      required: [true, 'Review comment is required'],
      trim: true,
      minLength: [10, 'Comment must be at least 10 characters'],
      maxLength: [1000, 'Comment cannot exceed 1000 characters'],
    },
    isVerifiedPurchase: {
      type: Boolean,
      default: false,
    },
  },
  { timestamps: true }
)

/* — Compound unique: one review per user per product — */
reviewSchema.index({ product: 1, user: 1 }, { unique: true })

/* — Post-save: update product ratings — */
reviewSchema.post('save', async function () {
  await updateProductRatings(this.product)
})

/* — Post-remove: update product ratings — */
reviewSchema.post('findOneAndDelete', async function (doc) {
  if (doc) await updateProductRatings(doc.product)
})
```

```

/* — Helper: recalculate average rating — */
async function updateProductRatings(productId) {
  const stats = await mongoose.model('Review').aggregate([
    { $match: { product: productId } },
    {
      $group: {
        _id: '$product',
        average: { $avg: '$rating' },
        count: { $sum: 1 },
      }
    }
  ])

  await mongoose.model('Product').findByIdAndUpdate(productId, {
    'ratings.average': stats.length > 0 ? Math.round(stats[0].average * 10) / 10 : 0,
    'ratings.count': stats.length > 0 ? stats[0].count : 0,
  })
}

const Review = mongoose.model('Review', reviewSchema)
export default Review

```

10.2 Verified Purchase Logic

```

// In review.controller.js – before saving review

const hasPurchased = await Order.findOne({
  user: req.user._id,
  'items.product': productId,
  orderStatus: 'delivered',
})

review.isVerifiedPurchase = !!hasPurchased

```

11. Coupon Schema

11.1 Full Schema Definition

```
// src/models/Coupon.model.js
import mongoose from 'mongoose'

const couponSchema = new mongoose.Schema(
  {
    code: {
      type: String,
      required: [true, 'Coupon code is required'],
      unique: true,
      uppercase: true,
      trim: true,
      minLength: [3, 'Code must be at least 3 characters'],
      maxLength: [20, 'Code cannot exceed 20 characters'],
    },
    discountType: {
      type: String,
      enum: ['percentage', 'flat'],
      required: [true, 'Discount type is required'],
    },
    discountValue: {
      type: Number,
      required: [true, 'Discount value is required'],
      min: [1, 'Discount value must be at least 1'],
    },
    minOrderAmount: {
      type: Number,
      default: 0,
    },
    maxDiscount: {
      type: Number, // Cap for percentage discounts
      default: null,
    },
    usagelimit: {
      type: Number,
      default: null, // null = unlimited
    },
    usedCount: {
      type: Number,
      default: 0,
    },
    perUserLimit: {
      type: Number,
      default: 1,
    },
    usedBy: [
      {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
      }
    ],
    isActive: {
      type: Boolean,
      default: true,
    },
  },

```

```

    expiresAt: {
      type: Date,
      required: [true, 'Expiry date is required'],
    },
  },
  { timestamps: true }
)

/* — Virtual: isExpired — */
couponSchema.virtual('isExpired').get(function () {
  return new Date() > this.expiresAt
})

/* — Virtual: isUsageLimitReached — */
couponSchema.virtual('isUsageLimitReached').get(function () {
  return this.usageLimit !== null && this.usedCount >= this.usageLimit
})

/* — Instance method: validate coupon for a user and order — */
couponSchema.methods.validateForUser = function (userId, orderAmount) {
  const errors = []

  if (!this.isActive) errors.push('This coupon is inactive')
  if (this.isExpired) errors.push('This coupon has expired')
  if (this.isUsageLimitReached) errors.push('This coupon usage limit has been reached')
  if (orderAmount < this.minOrderAmount)
    errors.push(`Minimum order amount of ₹${this.minOrderAmount} required`)

  const userUsageCount = this.usedBy.filter(id => id.toString() === userId.toString()).length
  if (userUsageCount >= this.perUserLimit)
    errors.push('You have already used this coupon')

  return errors
}

/* — Instance method: calculate discount amount — */
couponSchema.methods.calculateDiscount = function (orderAmount) {
  let discount = 0

  if (this.discountType === 'flat') {
    discount = Math.min(this.discountValue, orderAmount)
  } else {
    discount = (orderAmount * this.discountValue) / 100
    if (this.maxDiscount) {
      discount = Math.min(discount, this.maxDiscount)
    }
  }

  return Math.round(discount * 100) / 100
}

const Coupon = mongoose.model('Coupon', couponSchema)
export default Coupon

```

11.2 Coupon Examples

Code	Type	Value	Min Order	Max Discount	Per User
SAVE10	percentage	10%	₹500	₹200	1
FLAT50	flat	₹50	₹300	—	2
WELCOME20	percentage	20%	₹0	₹500	1
FREESHIP	flat	₹99	₹200	—	3

12. Database Indexes

```
// All indexes to be created (auto via schema + manual)

// — users —
db.users.createIndex({ email: 1 }, { unique: true })
db.users.createIndex({ role: 1 })
db.users.createIndex({ isBlocked: 1 })

// — products —
db.products.createIndex({ slug: 1 }, { unique: true })
db.products.createIndex({ category: 1 })
db.products.createIndex({ isActive: 1, isFeatured: 1 })
db.products.createIndex({ isActive: 1, price: 1 })
db.products.createIndex({ 'ratings.average': -1 })
db.products.createIndex({ stock: 1 })
db.products.createIndex({ name: 'text', description: 'text', tags: 'text', brand: 'text' })

// — categories —
db.categories.createIndex({ slug: 1 }, { unique: true })
db.categories.createIndex({ parent: 1 })
db.categories.createIndex({ isActive: 1 })

// — orders —
db.orders.createIndex({ orderNumber: 1 }, { unique: true })
db.orders.createIndex({ user: 1, createdAt: -1 })
db.orders.createIndex({ orderStatus: 1 })
db.orders.createIndex({ createdAt: -1 })

// — reviews —
db.reviews.createIndex({ product: 1, user: 1 }, { unique: true })
db.reviews.createIndex({ product: 1, createdAt: -1 })

// — coupons —
db.coupons.createIndex({ code: 1 }, { unique: true })
db.coupons.createIndex({ isActive: 1, expiresAt: 1 })

// — carts —
db.carts.createIndex({ user: 1 }, { unique: true })

// — wishlists —
db.wishlists.createIndex({ user: 1 }, { unique: true })
```

13. Schema Relationships

```
users (1) ————— (1) carts
users (1) ————— (1) wishlists
users (1) ————— (N) orders
users (1) ————— (N) reviews

products (1) ————— (N) reviews
products (N) ————— (1) categories
products (N) ————— (N) carts    [via cartItem.product]
products (N) ————— (N) wishlists [via wishlist.products[]]
products (N) ————— (N) orders    [via orderItem.product]

categories (1) ————— (N) products
categories (1) ————— (N) categories [self-ref: parent]

coupons (1) ————— (N) orders    [via order.coupon]
coupons (1) ————— (N) carts      [via cart.coupon]
coupons (N) ————— (N) users      [via coupon.usedBy[]]
```

Populate Patterns

```
// Product with category
Product.findById(id).populate('category', 'name slug image')

// Order with user + items.product
Order.findById(id)
  .populate('user', 'name email phone')
  .populate('items.product', 'name slug images')

// Review with user
Review.find({ product: id })
  .populate('user', 'name avatarUrl')
  .sort({ createdAt: -1 })

// Cart with product details
Cart.findOne({ user: userId })
  .populate('items.product', 'name price stock isActive images')
  .populate('coupon', 'code discountType discountValue')
```

14. Mongoose Model Files

File Structure Summary

```
src/models/
├─ User.model.js      ← users collection
├─ Product.model.js   ← products collection
├─ Category.model.js  ← categories collection
├─ Cart.model.js      ← carts collection
├─ Wishlist.model.js  ← wishlists collection
├─ Order.model.js     ← orders collection
├─ Review.model.js    ← reviews collection
└─ Coupon.model.js    ← coupons collection
```

Model Export Pattern

```
// Every model file ends with:
const ModelName = mongoose.model('ModelName', modelNameSchema)
export default ModelName

// Import in controllers:
import User      from '../models/User.model.js'
import Product   from '../models/Product.model.js'
import Order     from '../models/Order.model.js'
```

15. Seed Data

15.1 Seed Script Location

```
backend/  
└─ src/  
   └─ utils/  
      └─ seed.js      ← Run: node src/utils/seed.js
```

15.2 Seed Categories

```
const categories = [  
  { name: 'Electronics', slug: 'electronics' },  
  { name: 'Fashion', slug: 'fashion' },  
  { name: 'Home & Kitchen', slug: 'home-kitchen' },  
  { name: 'Books', slug: 'books' },  
  { name: 'Sports', slug: 'sports' },  
  { name: 'Beauty', slug: 'beauty' },  
  
  // Sub-categories  
  { name: 'Mobiles', slug: 'mobiles', parent: 'electronics' },  
  { name: 'Laptops', slug: 'laptops', parent: 'electronics' },  
  { name: 'Men\'s Wear', slug: 'mens-wear', parent: 'fashion' },  
  { name: 'Women\'s Wear', slug: 'womens-wear', parent: 'fashion' },  
]
```

15.3 Seed Admin User

```
const adminUser = {  
  name: 'Super Admin',  
  email: 'admin@shopease.com',  
  password: 'Admin@123456', // Hashed by pre-save hook  
  role: 'superadmin',  
}
```

15.4 Seed Products (Sample)

```
const sampleProducts = [
  {
    name:      'Wireless Bluetooth Headphones',
    price:     1299,
    originalPrice: 1999,
    stock:     50,
    isFeatured: true,
    // ... other fields
  },
  {
    name:      'Running Shoes',
    price:     2499,
    originalPrice: 3499,
    stock:     30,
    isFeatured: true,
  },
  // ... 8 more sample products
]
```

15.5 Seed Coupons

```
const seedCoupons = [
  {
    code:      'WELCOME20',
    discountType: 'percentage',
    discountValue: 20,
    maxDiscount: 500,
    perUserLimit: 1,
    expiresAt:  new Date('2027-01-01'),
  },
  {
    code:      'FLAT100',
    discountType: 'flat',
    discountValue: 100,
    minOrderAmount: 500,
    expiresAt:  new Date('2027-01-01'),
  },
]
```

Document History

Version	Date	Author	Notes
1.0	July 2026	Team	Initial Backend Schema Document

Previous Phase → [Phase 4: UI/UX Brief](#)

Next Phase → [Phase 6: API Documentation](#)